



K.R. Mangalam University
School of Engineering and Technology
LAB 2 BENCHMARKING REPORT
Design & Analysis of Algorithms
Course Code - ENCA351
(2026-27)

Submitted by:

Diksha Goel
2401201173
BCA (AI & DS) - A

Submitted To:

Dr. Aarti Sangwan
Faculty, SOET

Github Link - https://github.com/diksha29-goel/DAA_Lab2Assessment

1. Introduction

Asymptotic analysis tells us how the cost of an algorithm grows with input size, but it hides constant factors and implementation details. This lab builds a benchmarking tool that measures real execution time, memory use and operation counts, and then compares the measured behaviour with theoretical complexity. Three families of programs are studied: search algorithms, factorial implementations and loop-based programs.

Objectives

- Implement an automated benchmarking engine that records execution time, memory usage and operation counts for several input sizes.
- Prepare a complexity comparison table of theoretical and observed time complexity, space complexity and growth trends.
- Generate visualizations and comparative charts with performance observations.

2. Environment and Tools

Item	Details
Language	Python 3.12 (CPython)
Timing	time.perf_counter(), median of 7 repeats, batching for very fast calls
Memory	tracemalloc peak allocation (KB)
Operation counts	Manual instrumentation with an OpCounter object
Analysis and charts	NumPy (log-log regression), matplotlib

3. Methodology

Every algorithm is written with the signature `fn(input, counter)`. For each input size the engine (1) builds the input, (2) times the call, (3) measures peak memory in a separate pass with `tracemalloc`, and (4) counts operations in a third pass. Search algorithms were run on sorted lists of 1,000 to 1,000,000 integers with an absent target, which is the worst case. Factorial was run for $n = 50$ to 2000 and loop programs for sizes chosen so every run completes within a fraction of a second.

To compare theory and measurement, the growth exponent k in $T \propto n^k$ is estimated as the slope of a leastsquares line through $(\log n, \log T)$. The measured operation counts are compared with the exact theoretical formula $f(n)$ for each algorithm.

4. Implementation

4.1 Benchmarking engine

The engine is the class `BenchmarkEngine`. Its `run()` method loops over the input sizes, records a `Record` (suite, algorithm, n , median/min/std time, peak memory, operations, recursion depth) and prints a live log. `save()` exports all records to CSV and JSON. The core of the engine is shown below; the full program is in Appendix A.

```

@dataclass class BenchmarkEngine:
    repeats: int = 7
    records: List[Record] = field(default_factory=list)

    def _measure_time(self, call: Callable[[], None]) -> List[float]:
        """Median-of-N timing; fast calls are batched for timer
        accuracy."""
        t0 = time.perf_counter()
        call()

        single = max(time.perf_counter() - t0, 1e-9)
        inner = max(1,
int(2e-4 / single))
        # aim for >= ~0.2 ms per sample
        samples = []
        for in range(self.repeats):
            s = time.perf_counter()
            for in range(inner):
                call()
            samples.append((time.perf_counter() - s) / inner)
        return samples

    def run(self, suite: str, name: str, fn: Callable, sizes: List[int],
        Callable[[int], tuple]):
        """Benchmark `fn` for every input size in `sizes`."""
        for n in sizes:
            args = build_input(n)

            # --- time -----
            = self._measure_time(lambda: fn(*args, OpCounter()))

            # --- memory (separate pass; tracing slows execution) -----
            tracemalloc.start()
            tracemalloc.reset_peak()
            base, _ = tracemalloc.get_traced_memory()
            fn(*args, OpCounter())
            _, peak = tracemalloc.get_traced_memory()
            tracemalloc.stop()

            # --- operation count -----
            c = OpCounter()
            fn(*args, c)

            self.records.append(Record(
                suite=suite,
                algorithm=name, n=n,
                time ms=statistics.median(samples) * 1e3,
                time_min_ms=min(samples) * 1e3,
                time_std_ms=(statistics.pstdev(samples) * 1e3),
                memory kb=max(peak - base, 0) / 1024,
                operations=c.ops,
                max_depth=c.max_depth,
            ))
            r = self.records[-1]
            print(f" [{suite:9s}] {name:22s} n={n:>9,d} "
                f"time={r.time ms:10.4f} ms mem={r.memory kb:10.2f} KB "
                f"ops={r.operations:>12,d}")

```

4.2 Algorithms implemented

Group	Algorithms	Counted operation
Search	Linear, Binary, Jump search	Element comparisons
Factorial	Iterative, Recursive, Memoized recursive, math.factorial	Multiplications (1 call for built-in); recursion depth

Loops	Single, Nested, Dependent ($j < i$), Triple, Logarithmic ($i *= 2$)	Loop iterations
-------	---	-----------------

4.3 Sample output

```
$ python benchmark_engine.py
== Search algorithms (worst case) ==

[search ] Linear Search      n=   1,000 time=   0.0228 ms mem=   0.16 KB ops=    1,000
[search ] Linear Search      n=   5,000 time=   0.1266 ms mem=   0.16 KB ops=    5,000
[search ] Linear Search      n=  10,000 time=   0.2479 ms mem=   0.16 KB ops=   10,000
[search ] Linear Search      n=  50,000 time=   1.2123 ms mem=   0.16 KB ops=   50,000
[search ] Linear Search      n= 100,000 time=   2.4250 ms mem=   0.16 KB ops=  100,000
[search ] Linear Search      n= 500,000 time=  12.1897 ms mem=   0.16 KB ops=  500,000
[search ] Linear Search      n=1,000,000 time=  24.9113 ms mem=   0.16 KB ops= 1,000,000
[search ] Binary Search      n=   1,000 time=   0.0017 ms mem=   0.15 KB ops=         9
[search ] Binary Search      n=   5,000 time=   0.0022 ms mem=   0.18 KB ops=        12
[search ] Binary Search      n=  10,000 time=   0.0023 ms mem=   0.18 KB ops=        13
[search ] Binary Search      n=  50,000 time=   0.0026 ms mem=   0.18 KB ops=        15
[search ] Binary Search      n= 100,000 time=   0.0027 ms mem=   0.18 KB ops=        16
[search ] Binary Search      n= 500,000 time=   0.0030 ms mem=   0.18 KB ops=        18
[search ] Binary Search      n=1,000,000 time=   0.0031 ms mem=   0.18 KB ops=        19
[search ] Jump Search        n=   1,000 time=   0.0029 ms mem=   0.17 KB ops=        31
[search ] Jump Search        n=   5,000 time=   0.0045 ms mem=   0.17 KB ops=       70
[search ] Jump Search        n=  10,000 time=   0.0059 ms mem=   0.17 KB ops=      100
[search ] Jump Search        n=  50,000 time=   0.0114 ms mem=   0.17 KB ops=     223
[search ] Jump Search        n= 100,000 time=   0.0177 ms mem=   0.26 KB ops=     316
[search ] Jump Search        n= 500,000 time=   0.0455 ms mem=   0.26 KB ops=    707
[search ] Jump Search        n=1,000,000 time=   0.0697 ms mem=   0.26 KB ops=   1,000

== Factorial implementations ==

[factorial] Iterative        n=     50 time=   0.0042 ms mem=   0.20 KB ops=        49
[factorial] Iterative        n=    100 time=   0.0087 ms mem=   0.29 KB ops=       99
[factorial] Iterative        n=    200 time=   0.0199 ms mem=   0.48 KB ops=      199
```

Figure 1: Console output of the benchmarking engine (excerpt).

5. Complexity Comparison Table

Table 1 summarises the theoretical and observed time complexity, space complexity and growth trend of each implemented algorithm.

Algorithm	Theory: time	Theory: space	Observed time slope	Ops at max n	Peak mem (KB)	Growth trend
Linear Search	$O(n)$	$O(1)$	1.01 (≈ 1)	1,000,000	0.16	Linear: time and comparisons grow in direct proportion to n
Binary Search	$O(\log n)$	$O(1)$	0.08 ($\approx \sim 0 (\log)$)	19	0.18	Logarithmic: $n \times 1000$ adds only ~ 10 comparisons (needs sorted input)
Jump Search	$O(\sqrt{n})$	$O(1)$	0.47 (≈ 0.5)	1,000	0.26	Square-root: between linear and logarithmic (needs sorted input)
Iterative	$O(n)$ mults*	$O(1)$ aux	1.41 (≈ 1)	1,999	5.17	Linear in operations; time slightly super-linear (big-integer multiplication)
Recursive	$O(n)$ mults*	$O(n)$	1.28 (≈ 1)	1,999	55.11	Linear ops; call stack grows linearly (depth = n)

Memoized Recursive	$O(n)$ mults*	$O(n)$	1.27 (≈ 1)	1,999	2,477.52	Linear ops; stack AND cache grow with n ; cache holds every intermediate result
Built-in math.factorial	$O(1)$ call*	$O(1)$ aux	1.51 (≈ 0 (const))	1	13.18	Single C-level call; fastest, only big-integer cost remains
Single Loop	$O(n)$	$O(1)$	1.01 (≈ 1)	500,000	0.23	Linear
Nested Loop (n^2)	$O(n^2)$	$O(1)$	2.05 (≈ 2)	1,000,000	0.34	Quadratic: doubling n quadruples the work
Dependent Loop ($n^2/2$)	$O(n^2)$	$O(1)$	2.07 (≈ 2)	499,500	0.34	Quadratic with constant $1/2$ ($n(n-1)/2$ iterations)
Algorithm	Theory: time	Theory: space	Observed time slope	Ops at max n	Peak mem (KB)	Growth trend
Triple Loop (n^3)	$O(n^3)$	$O(1)$	3.05 (≈ 3)	1,728,000	0.28	Cubic: doubling n multiplies the work by 8
Logarithmic Loop	$O(\log n)$	$O(1)$	0.05 (≈ 0 (log))	60	0.16	Logarithmic: i doubles every pass ($\approx \log_2 n$ iterations)

Table 1: Theoretical vs observed performance. Operation counts equal the theoretical formulas exactly ($\text{ops} \div f(n) = 1.000$).

* For factorial, the operation count is exactly $n - 1$ multiplications for every implementation, but each multiplication works on a growing big integer, so measured time grows a little faster than linearly (observed slopes $1.3 - 1.5$). Observed slope = gradient of $\log(\text{time})$ vs $\log(n)$; a slope of k means time $\propto n^k$. Logarithmic algorithms show slopes close to 0 on a log-log plot.

6. Visualizations and Performance Observations

6.1 Search algorithms

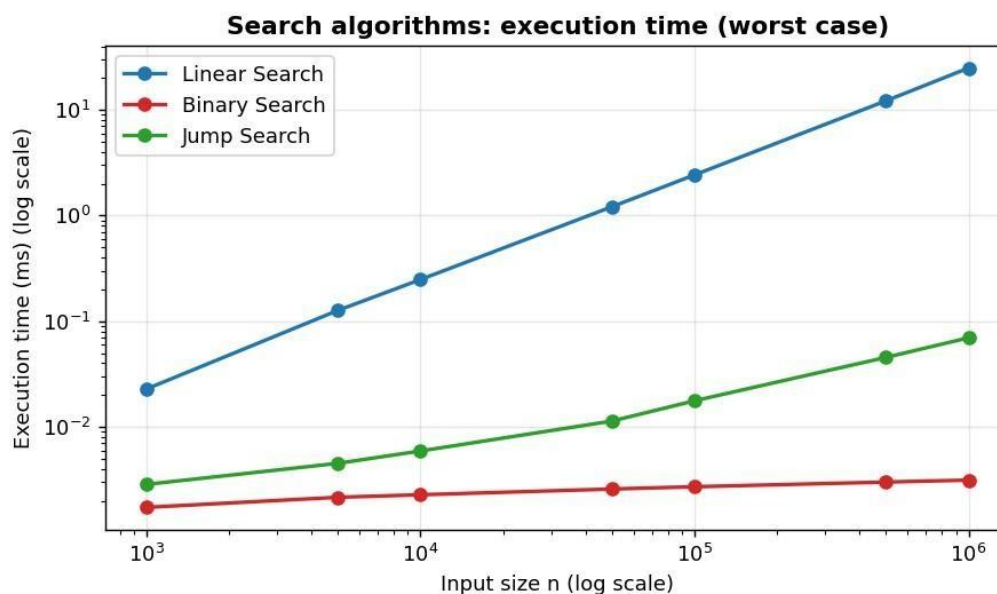


Figure 2: Execution time of the search algorithms (worst case).

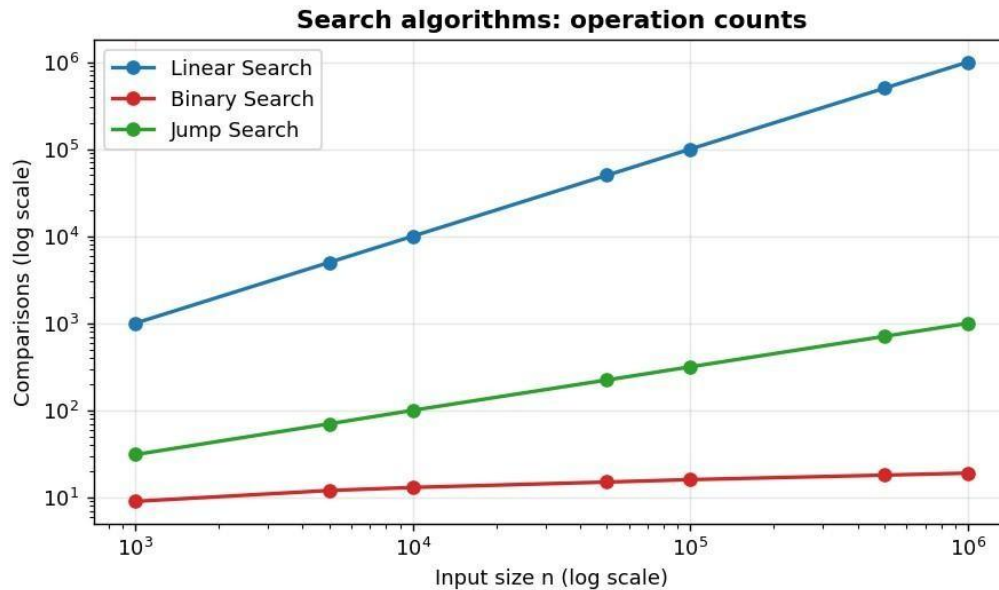


Figure 3: Comparisons performed by the search algorithms.

- **Linear search scales linearly.** Comparisons grew exactly with n (1,000,000 comparisons for $n = 1,000,000$) and the time slope was 1.01. At $n = 1,000,000$ it took 24.911 ms.
- **Binary search is almost flat.** It needed only 19 comparisons for one million elements ($\approx \log_2 n$) and 0.0031 ms, about 7,942× faster than linear search at that size.
- **Jump search sits in between.** 1,000 comparisons ($= \sqrt{n}$) and 0.070 ms, with a measured slope of 0.47, which matches the theoretical 0.5.
- **Memory is constant for all three.** Each uses only a few index variables (peak ≈ 0.2 KB), confirming $O(1)$ auxiliary space.
- **Trade-off.** Binary and jump search need sorted data; for a one-off search of unsorted data, linear search avoids the cost of sorting.

6.2 Factorial implementations

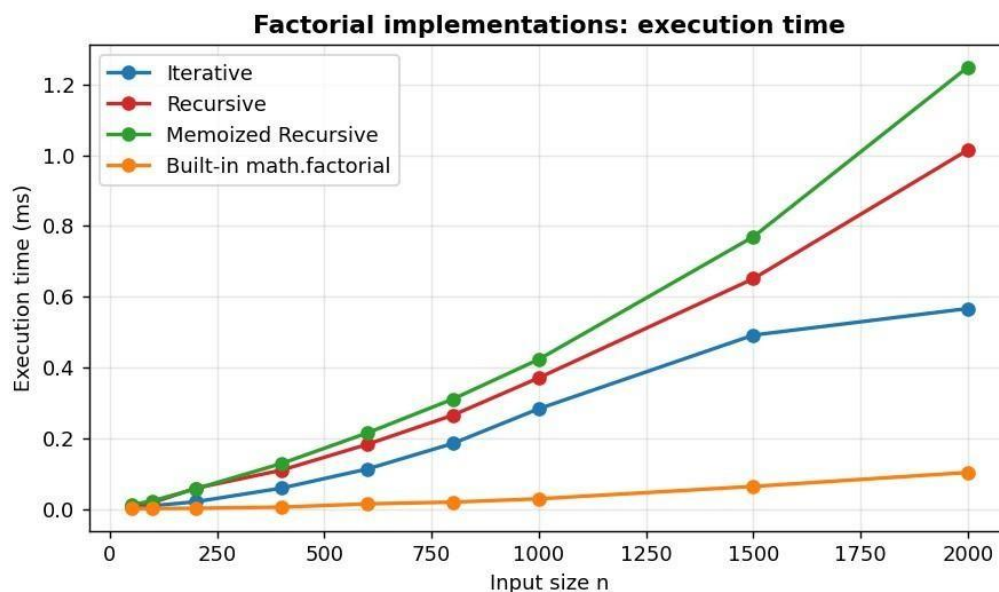


Figure 4: Execution time of the factorial implementations.

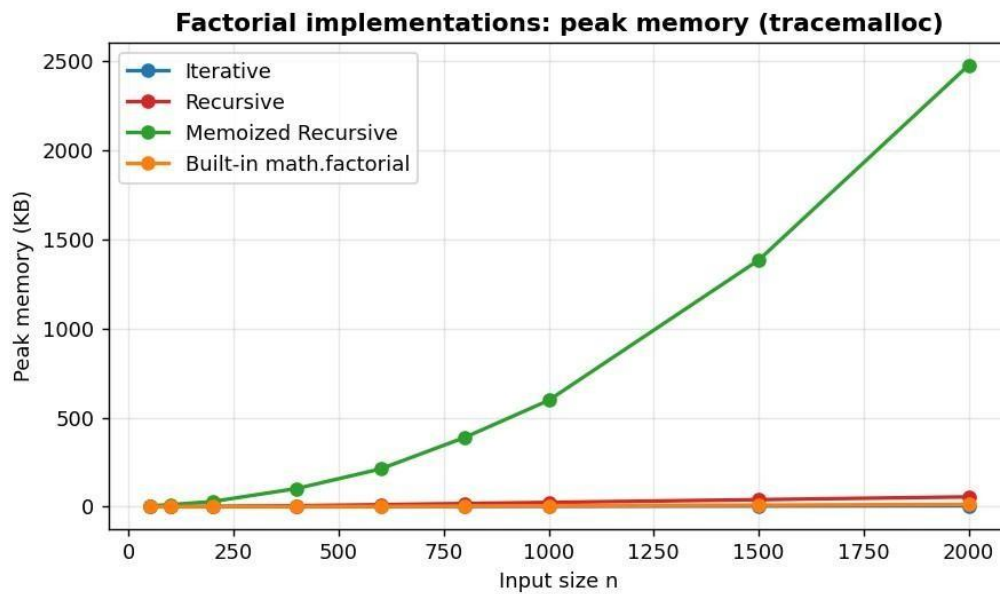


Figure 5: Peak memory of the factorial implementations.

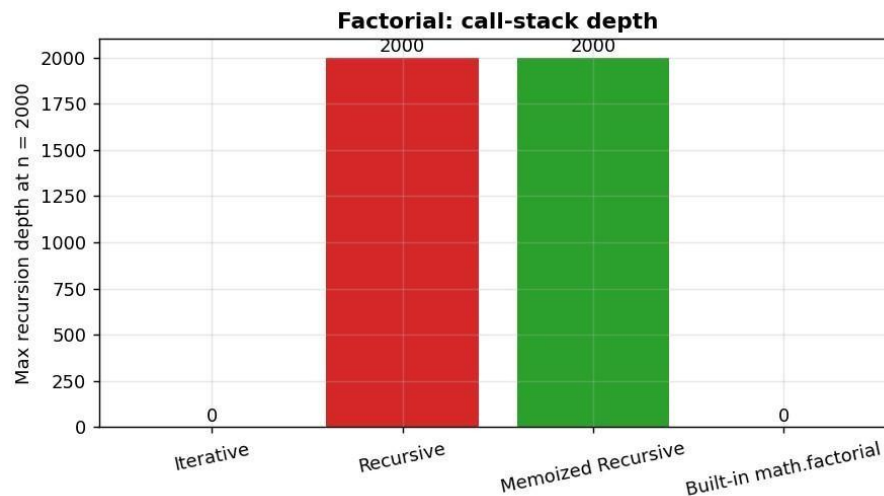


Figure 6: Maximum recursion depth at $n = 2000$.

- **Same operation count, different cost.** Iterative, recursive and memoized versions all perform exactly $n - 1$ multiplications, yet at $n = 2000$ they took 0.567 ms, 1.015 ms and 1.249 ms respectively. Function-call overhead makes recursion slower than a loop.
- **Built-in is fastest.** `math.factorial` took 0.103 ms at $n = 2000$, about 5.5× faster than the iterative loop, because the whole loop runs in optimised C code.
- **Time is slightly super-linear.** Observed slopes were 1.41 (iterative), 1.28 (recursive) and 1.27 (memoized) rather than 1.0: factorial values become huge integers, and multiplying big integers costs more than multiplying small ones.
- **Space differs the most.** Recursion reaches a call-stack depth of 2,000 at $n = 2000$ while the iterative version needs none ($O(n)$ vs $O(1)$ stack). The memoized version stored every intermediate factorial and used 2.42 MB at $n = 2000$; doubling n from 1000 to 2000 multiplied its memory by 4.2, i.e. roughly quadratic growth because the cached numbers are themselves growing.

- **Memoization did not help here.** Each top-level call starts with an empty cache and every sub-problem is visited once, so memoization only adds overhead. It pays off only when the same function is called repeatedly, e.g. computing many factorials in sequence.

6.3 Loop-based programs

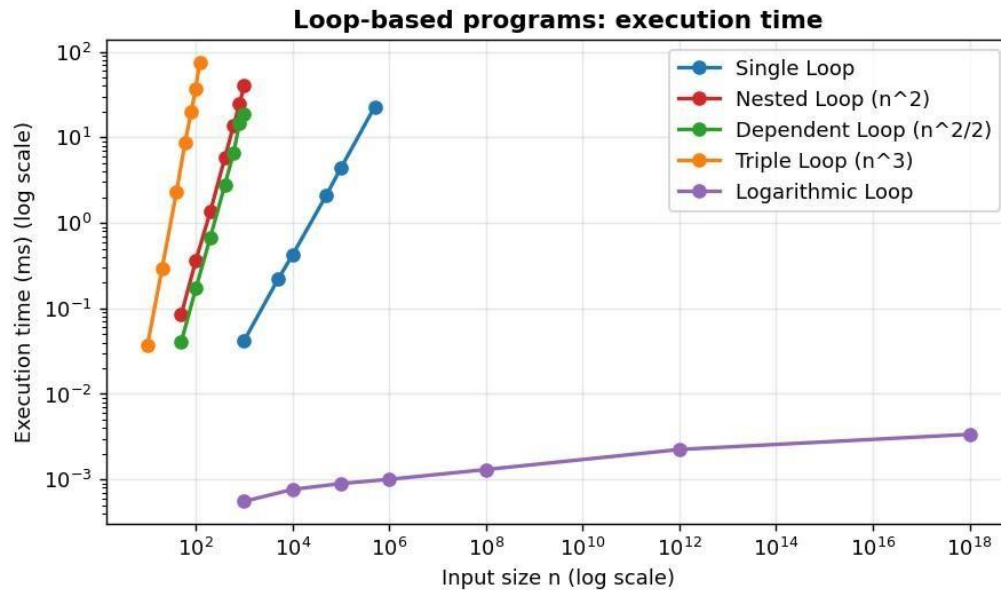


Figure 7: Execution time of loop-based programs.

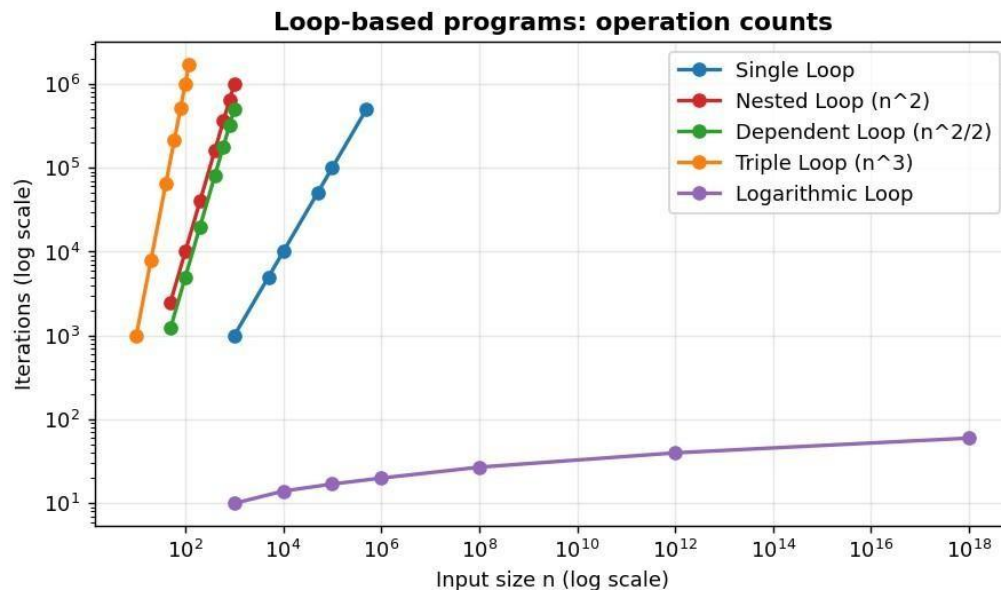


Figure 8: Iteration counts of loop-based programs.

- **Operation counts match theory exactly.** Single loop = n , nested = n^2 , dependent = $n(n-1)/2$ and triple = n^3 iterations (ops/f(n) = 1.000 for all). The logarithmic loop ran only 60 iterations even for $n = 10^{18}$.
- **Time follows the same exponents.** Observed log-log slopes were 1.01, 2.05, 2.07, 3.05 and 0.05 for single, nested, dependent, triple and logarithmic loops, against theoretical 1, 2, 2, 3 and ≈ 0 .

-
- **Constant factors matter.** The dependent loop does half the iterations of the nested loop at $n = 1000$ (499,500 vs 1,000,000) and ran in 19.0 ms vs 40.4 ms. Both are $O(n^2)$, but the constant differs.
Practical impact. The triple loop at only $n = 120$ (1.7 million iterations) took 76.2 ms, longer than a single loop of 500,000 iterations (22.8 ms), showing how fast polynomial growth overtakes linear growth.
- **Memory is constant.** All loop programs use $O(1)$ extra space (peak < 0.4 KB).

6.4 Overall comparison

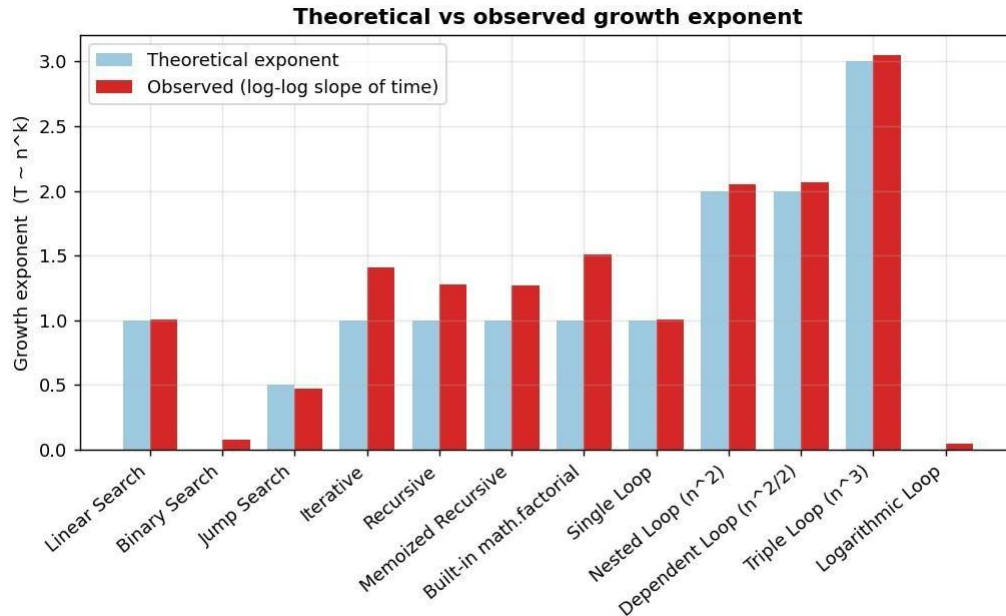


Figure 9: Theoretical exponent vs observed slope.

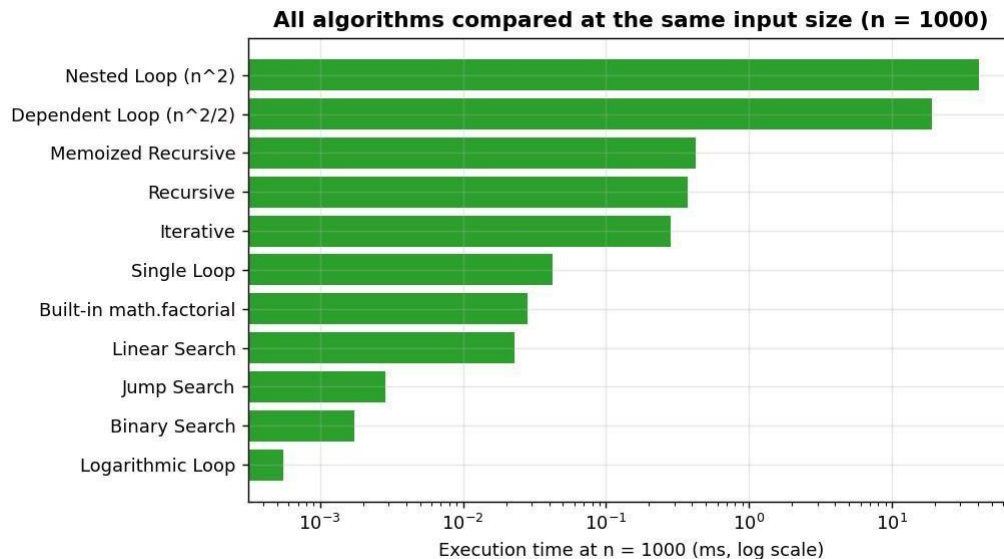


Figure 10: All algorithms at $n = 1000$.

-

7. Discussion

- Theory and experiment agree. Observed growth exponents match the theoretical complexity classes for all search and loop programs, and operation counts match the formulas exactly.
- Algorithm choice dominates. At one million elements binary search is thousands of times faster than linear search; a triple loop becomes slow at n of only a few hundred.
Time and space trade off. Memoized factorial reduces nothing for a single call but costs the most memory; recursion adds stack depth that the iterative loop avoids.
- Operation counts and wall-clock time answer different questions. Counts are machine-independent and verify the theory, while time includes real effects such as big-integer arithmetic and interpreter overhead.

8. Limitations

- Timing noise: at microsecond scale (binary search, logarithmic loop) the timer, CPU caches and OS scheduling add variation, so slopes for $O(\log n)$ algorithms are close to, but not exactly, zero.
- Instrumentation overhead: operation counters add a small constant cost to every measured call. This does not change growth rates but slightly inflates absolute times.
- Memory measurement: tracemalloc only tracks memory allocated through Python's allocator; it does not include the interpreter stack frames, so recursion depth is reported separately.
- Language effects: results come from CPython, where interpreter overhead and big-integer arithmetic affect constants. Absolute values depend on the machine and will differ on another computer.
- Worst-case inputs were used for the searches (absent target); average-case and best-case behaviour were not measured.

9. Conclusion

The automated benchmarking engine successfully recorded execution time, memory usage and operation counts for twelve algorithms across multiple input sizes. The results confirm $O(n)$, $O(\log n)$, $O(\sqrt{n})$, $O(n^2)$ and $O(n^3)$ growth for the search and loop programs, and reveal the practical differences between iterative, recursive, memoized and built-in factorials. Combining theoretical analysis with empirical measurement gives a more complete picture of algorithm performance than either approach alone.

Appendix A: Source code(benchmark_engine.py)

```
"""
Lab 2: Algorithm Performance Measurement and Benchmarking Tool
=====

An automated benchmarking engine that runs algorithms / code snippets for multiple input
sizes and records:

*      execution time      (time.perf_counter, median of several repeats)
*      memory usage       (tracemalloc peak allocation during the call)
*      operation counts    (instrumented comparisons / iterations / calls)      * recursion depth
(extra "space" indicator for recursive code)

Results are written to CSV + JSON and later turned into charts / tables.
Usage:      python benchmark_engine.py      # runs every suite, saves
results/ ""  import csv import json import math import os import statistics
import sys import time import tracemalloc
from dataclasses import dataclass, field, asdict from typing
import Callable, Dict, List

sys.setrecursionlimit(10_000)

# -----
# 1. Instrumentation: operation counter
# ----- class OpCounter:
#      """Counts basic operations (comparisons, iterations, calls)."""      slots
#      = ("ops", "depth", "max_depth")

def __init__(self):
self.ops = 0      self.depth =
0      self.max_depth = 0

# ----- #
2. Algorithms under test
# -----
# ---- Search algorithms (worst case: target is absent) -----
def linear_search(arr, target, c: OpCounter):      for x in arr:
c.ops += 1      # one comparison per element      if x
== target:      return True      return False

def binary_search(arr, target, c: OpCounter):
lo, hi = 0, len(arr) - 1      while lo <= hi:
mid = (lo + hi) // 2
c.ops += 1      # one comparison per halving step      if
arr[mid] == target:
return True
if arr[mid] < target:
lo = mid + 1
else:
hi =
mid - 1      return False

def jump_search(arr, target, c: OpCounter):
```

```

    n = len(arr)
    step = int(math.sqrt(n))    prev = 0    while prev < n
and arr[min(prev + step, n) - 1] < target:        c.ops += 1
prev += step    for i in range(prev, min(prev + step, n)):
c.ops += 1        if arr[i] == target:            return
True            return False

# ---- Factorial implementations ----- def
fact_iterative(n, c: OpCounter):    result = 1    for i in range(2, n +
1):
        c.ops += 1        # one multiplication per
iteration    result *= i    return result

def fact_recursive(n, c: OpCounter):    c.depth
+= 1
    c.max_depth = max(c.max_depth, c.depth)    if
n <= 1:
        c.depth -= 1    return
1
    c.ops += 1        # one multiplication per
call    r = n * fact_recursive(n - 1, c)    c.depth -= 1    return
r

def fact_memoized(n, c: OpCounter, _cache=None):    if
    cache is None:
        cache = {0: 1, 1: 1}        # fresh cache for every top-level call    c.depth
+= 1
    c.max_depth = max(c.max_depth, c.depth)    if
n in cache:
        c.depth -= 1
return _cache[n]    c.ops +=
1
    cache[n] = n * fact_memoized(n - 1, c, cache)    c.depth
-= 1    return _cache[n]

def fact_builtin(n, c: OpCounter):
    c.ops += 1        # one call into optimised C code    return
math.factorial(n)

# ---- Loop-based programs ----- def loop_single(n,
c: OpCounter):
    total = 0    for i
in range(n):
c.ops += 1
total += i    return
total

def loop_nested(n, c: OpCounter):
total = 0    for i in range(n):
for j in range(n):
c.ops += 1        total
+= 1    return
total

def loop_dependent(n, c:
OpCounter):    total = 0    for i
in range(n):
for j in
range(i):

```

```

        c.ops += 1
        total
+= 1    return total

def loop_triple(n, c: OpCounter):
    total = 0
    for i in range(n):
        for j in range(n):
            for k
            in range(n):
                c.ops += 1
                total
+= 1    return total

def loop_logarithmic(n, c:
OpCounter):
    i, total = 1, 0
    while i < n:
        c.ops += 1
        total += i
        i *=
        2
    return
total

# -----
# 3. Benchmark engine
# -----

@dataclass class
Record:
    suite: str
algorithm: str
    n: int
time_ms: float
time_min_ms: float
time_std_ms: float
memory_kb: float
operations: int
max_depth: int

@dataclass class BenchmarkEngine:
    repeats: int = 7
    records: List[Record] = field(default_factory=list)

    def _measure_time(self, call: Callable[[], None]) -> List[float]:
        """Median-of-N timing; fast calls are batched for timer accuracy."""
        t0 = time.perf_counter()
        call()
        single = max(time.perf_counter() - t0, 1e-9)
        inner = max(1,
int(2e-4 / single))
        # aim for >= ~0.2 ms per sample
        samples = []
        for _ in range(self.repeats):
            s = time.perf_counter()
            for _ in range(inner):
                call()
            samples.append((time.perf_counter() - s) / inner)
        return
samples

    def run(self, suite: str, name: str, fn: Callable, sizes: List[int],
Callable[[int], tuple]):
        """Benchmark `fn` for every input size in `sizes`."""
        for
n in sizes:
            args = build_input(n)

            # --- time -----
            samples
= self._measure_time(lambda: fn(*args, OpCounter()))

            # --- memory (separate pass; tracing slows execution) -----
            tracemalloc.start()
            tracemalloc.reset_peak()
            base,
            = tracemalloc.get_traced_memory()
            fn(*args, OpCounter())
            _, peak = tracemalloc.get_traced_memory()
            tracemalloc.stop()

```

```

# --- operation count -----
c = OpCounter()          fn(*args, c)

        self.records.append(Record(          suite=suite,
algorithm=name, n=n,
time_ms=statistics.median(samples) * 1e3,
time_min_ms=min(samples) * 1e3,
        time_std_ms=(statistics.pstdev(samples) * 1e3),
memory_kb=max(peak - base, 0) / 1024,          operations=c.ops,
max_depth=c.max_depth,
        ))
    r = self.records[-1]
    print(f" [{suite:9s}] {name:22s} n={n:>9,d} "
          f"time={r.time_ms:10.4f} ms mem={r.memory_kb:10.2f} KB "
f"ops={r.operations:>12,d}")

# ---- persistence -----
-   def save(self, folder: str):          os.makedirs(folder,
exist_ok=True)          rows = [asdict(r) for r in self.records]
with open(os.path.join(folder, "results.json"), "w") as f:
    json.dump(rows, f, indent=2)          with open(os.path.join(folder,
"results.csv"), "w", newline="") as f:          w = csv.DictWriter(f,
fieldnames=list(rows[0].keys()))          w.writeheader()
        w.writerows(rows)

# ----- #
4. Benchmark configuration
# ----- def sorted_input(n):

    return (list(range(n)), -1)          # absent target => worst case

def int_input(n):
    return (n,)

def run_all(out_dir="results"):
    eng = BenchmarkEngine(repeats=7)

    print("\n== Search algorithms (worst case) ==")
    search_sizes = [1_000, 5_000, 10_000, 50_000, 100_000, 500_000, 1_000_000]
    eng.run("search", "Linear Search", linear_search, search_sizes, sorted_input)
    eng.run("search", "Binary
Search", binary_search, search_sizes, sorted_input)          eng.run("search", "Jump Search",
jump_search, search_sizes, sorted_input)

    print("\n== Factorial implementations ==")
    fact_sizes = [50, 100, 200, 400, 600, 800, 1000, 1500, 2000]
    eng.run("factorial", "Iterative", fact_iterative, fact_sizes, int_input)
    eng.run("factorial", "Recursive", fact_recursive, fact_sizes, int_input)          eng.run("factorial",
"Memoized Recursive", fact_memoized, fact_sizes, int_input)          eng.run("factorial", "Built-in
math.factorial", fact_builtin, fact_sizes, int_input)

    print("\n== Loop-based programs ==")
    eng.run("loops", "Single Loop", loop_single,
        [1_000, 5_000, 10_000, 50_000, 100_000, 500_000],
int_input)          eng.run("loops", "Nested Loop (n^2)", loop_nested,
[50, 100, 200, 400, 600, 800, 1000], int_input)          eng.run("loops",
"Dependent Loop (n^2/2)", loop_dependent,          [50, 100, 200,
400, 600, 800, 1000], int_input)          eng.run("loops", "Triple Loop (n^3)",
loop_triple,          [10, 20, 40, 60, 80, 100, 120], int_input)
    eng.run("loops", "Logarithmic Loop", loop_logarithmic,          [10**3,
10**4, 10**5, 10**6, 10**8, 10**12, 10**18], int_input)

```

```
eng.save(out_dir)
print(f"\nSaved {len(eng.records)} records to '{out_dir}/'")

return eng

if __name__ == "__main__":
    run_all()
```

Appendix B: Complete measurement data

Algorithm	n	Time (ms)	Memory (KB)	Operations	Depth
Linear Search	1,000	0.0228	0.16	1,000	0
Linear Search	5,000	0.1266	0.16	5,000	0
Linear Search	10,000	0.2479	0.16	10,000	0
Linear Search	50,000	1.2123	0.16	50,000	0
Linear Search	100,000	2.4250	0.16	100,000	0
Linear Search	500,000	12.1897	0.16	500,000	0
Linear Search	1,000,000	24.9113	0.16	1,000,000	0
Binary Search	1,000	0.0017	0.15	9	0
Binary Search	5,000	0.0022	0.18	12	0
Binary Search	10,000	0.0023	0.18	13	0
Binary Search	50,000	0.0026	0.18	15	0
Binary Search	100,000	0.0027	0.18	16	0
Binary Search	500,000	0.0030	0.18	18	0
Binary Search	1,000,000	0.0031	0.18	19	0
Jump Search	1,000	0.0029	0.17	31	0
Jump Search	5,000	0.0045	0.17	70	0
Jump Search	10,000	0.0059	0.17	100	0
Jump Search	50,000	0.0114	0.17	223	0
Jump Search	100,000	0.0177	0.26	316	0
Jump Search	500,000	0.0455	0.26	707	0
Jump Search	1,000,000	0.0697	0.26	1,000	0
Iterative	50	0.0042	0.20	49	0
Iterative	100	0.0087	0.29	99	0
Iterative	200	0.0199	0.48	199	0
Iterative	400	0.0585	0.96	399	0
Iterative	600	0.1125	1.43	599	0
Iterative	800	0.1850	1.92	799	0
Iterative	1,000	0.2830	2.43	999	0
Iterative	1,500	0.4913	3.77	1,499	0
Iterative	2,000	0.5665	5.17	1,999	0

Recursive	50	0.0087	0.16	49	50
Recursive	100	0.0175	0.25	99	100
Recursive	200	0.0581	0.44	199	200
Recursive	400	0.1092	5.08	399	400
Recursive	600	0.1822	11.36	599	600
Recursive	800	0.2649	17.61	799	800
Recursive	1,000	0.3704	23.86	999	1000
Recursive	1,500	0.6508	39.49	1,499	1500
Recursive	2,000	1.0148	55.11	1,999	2000
Memoized Recursive	50	0.0114	4.74	49	50
Memoized Recursive	100	0.0226	11.07	99	100
Memoized Recursive	200	0.0564	29.09	199	200

Algorithm	n	Time (ms)	Memory (KB)	Operations	Depth
Memoized Recursive	400	0.1279	101.75	399	400
Memoized Recursive	600	0.2145	211.96	599	600
Memoized Recursive	800	0.3105	388.20	799	800
Memoized Recursive	1,000	0.4227	596.61	999	1000
Memoized Recursive	1,500	0.7694	1,383.55	1,499	1500
Memoized Recursive	2,000	1.2492	2,477.52	1,999	2000
Built-in math.factorial	50	0.0005	0.21	1	0
Built-in math.factorial	100	0.0008	0.29	1	0
Built-in math.factorial	200	0.0017	0.41	1	0
Built-in math.factorial	400	0.0048	0.81	1	0
Built-in math.factorial	600	0.0139	1.25	1	0
Built-in math.factorial	800	0.0191	3.29	1	0
Built-in math.factorial	1,000	0.0283	5.47	1	0
Built-in math.factorial	1,500	0.0634	8.55	1	0
Built-in math.factorial	2,000	0.1029	13.18	1	0
Single Loop	1,000	0.0420	0.22	1,000	0
Single Loop	5,000	0.2187	0.22	5,000	0
Single Loop	10,000	0.4213	0.22	10,000	0

Single Loop	50,000	2.1095	0.23	50,000	0
Single Loop	100,000	4.3805	0.23	100,000	0
Single Loop	500,000	22.7978	0.23	500,000	0
Nested Loop (n^2)	50	0.0845	0.24	2,500	0
Nested Loop (n^2)	100	0.3622	0.24	10,000	0
Nested Loop (n^2)	200	1.3610	0.24	40,000	0
Nested Loop (n^2)	400	5.8254	0.34	160,000	0
Nested Loop (n^2)	600	13.5971	0.34	360,000	0
Nested Loop (n^2)	800	24.5398	0.34	640,000	0
Nested Loop (n^2)	1,000	40.3517	0.34	1,000,000	0
Dependent Loop ($n^2/2$)	50	0.0406	0.24	1,225	0
Dependent Loop ($n^2/2$)	100	0.1704	0.24	4,950	0
Dependent Loop ($n^2/2$)	200	0.6807	0.24	19,900	0
Dependent Loop ($n^2/2$)	400	2.8022	0.34	79,800	0
Dependent Loop ($n^2/2$)	600	6.5081	0.34	179,700	0
Dependent Loop ($n^2/2$)	800	14.5244	0.34	319,600	0
Dependent Loop ($n^2/2$)	1,000	18.9793	0.34	499,500	0
Triple Loop (n^3)	10	0.0370	0.28	1,000	0
Triple Loop (n^3)	20	0.2910	0.28	8,000	0
Triple Loop (n^3)	40	2.3130	0.28	64,000	0
Triple Loop (n^3)	60	8.7445	0.28	216,000	0
Triple Loop (n^3)	80	19.6728	0.28	512,000	0
Triple Loop (n^3)	100	37.5396	0.28	1,000,000	0
Triple Loop (n^3)	120	76.2457	0.28	1,728,000	0
Logarithmic Loop	1,000	0.0006	0.15	10	0
Algorithm	n	Time (ms)	Memory (KB)	Operations	Depth
Logarithmic Loop	10,000	0.0008	0.15	14	0
Logarithmic Loop	100,000	0.0009	0.15	17	0
Logarithmic Loop	1,000,000	0.0010	0.15	20	0
Logarithmic Loop	100,000,000	0.0013	0.15	27	0
Logarithmic Loop	1,000,000,000,000	0.0022	0.16	40	0
Logarithmic Loop	1,000,000,000,000,000,000	0.0034	0.16	60	0

